Stage 2: Speaker Diarization with pyannote.audio

Theory notes and interview questions
Project: Doctor–Patient Conversation Audio -> SOAP Notes

What this stage does: answers "who spoke when?" It takes the same audio file as Stage 1 and outputs anonymous speaker labels (SPEAKER_00, SPEAKER_01, ...) with start/end times. It does not look at any text, and the labels do **not** mean doctor or patient.

Contents

1. What diarization is (and is not)
2. Why diarization is hard
3. The pyannote pipeline at a glance
4. Step 1 - Local segmentation (powerset)
5. Step 2 - Speaker embeddings (voiceprints)
6. Step 3 - Global clustering (VBx + PLDA)
7. Step 4 - Reconstruction and speaker counting
8. Regular vs exclusive diarization
9. pyannote.audio 4.x vs 3.x
10. Hugging Face gated models and tokens
11. Our code, step by step
12. Reading our output
13. Measuring quality: DER
14. Limitations in a clinical setting
15. Interview questions and answers

1. What diarization is (and is not)

Task	Question	Output
Speech-to-text (Stage 1)	What was said, and when?	Text + timestamps
Speaker diarization (Stage 2)	**Who spoke when?**	Anonymous labels + time intervals
Speaker identification	Is this voice person X?	A known name (needs enrolled voice samples)
Speaker verification	Are these two clips the same person?	Yes / no + score
Role identification (Stage 4)	Which label is the doctor?	SPEAKER_00 = Doctor, etc.

- Diarization is **unsupervised** with respect to identity: it never knows who people are, only that segments sound like the same or different voices.
- Label numbers are **arbitrary**: SPEAKER_00 is just 'the first cluster'. Running a different file, the doctor might be SPEAKER_01. Mapping labels to roles is a later stage (using what is said, e.g. 'What brings you in today?' sounds like a doctor).
- It uses only acoustics (voice characteristics), not words.

2. Why diarization is hard

- **Unknown number of speakers** - the system must decide how many voices exist.
- **Overlapping speech** - in real consultations people interrupt and say 'mm-hm' while the other talks.
- **Short turns** - a 0.3 s 'yes' carries very little voice information.
- **Similar voices** - two speakers of the same gender/age/accent are easy to confuse.
- **Recording conditions** - distance to the microphone, room echo, background noise and phone codecs change how a voice sounds.

3. The pyannote pipeline at a glance

pyannote/speaker-diarization-community-1 is not one neural network - it is a **pipeline** of two neural models plus a clustering algorithm:

```
16 kHz mono audio
   |
   v
[1] SEGMENTATION model  (sliding 10 s windows)
      for each ~17 ms frame: who is active? (local speakers A/B/C, overlap)
   v
[2] EMBEDDING model  (WeSpeaker ResNet34)
      one 256-number 'voiceprint' per (window, local speaker)
   v
[3] CLUSTERING  (VBx with PLDA)
      group voiceprints of the same person -> global SPEAKER_00, SPEAKER_01 ...
   v
[4] RECONSTRUCTION
      stitch windows together, count speakers per frame -> final speaker turns
```

This design is called **hybrid diarization**: a neural end-to-end model handles short windows (where it can model overlap well), and clustering links speakers across the whole recording (which end-to-end

models struggle with for long audio).

4. Step 1 - Local segmentation (powerset)

- The audio is cut into **10-second windows** that overlap (a sliding window with a small step), so every moment is seen several times.
- The segmentation model (SincNet filters on raw waveform -> LSTM layers -> classifier) outputs, for each frame of about 17 ms, which **local** speakers are active.
- It uses a **powerset** encoding: instead of 3 independent yes/no outputs, it predicts one class out of 7 combinations: {nobody}, {A}, {B}, {C}, {A+B}, {A+C}, {B+C}. At most 3 speakers per window and at most 2 talking at once. This makes overlap detection a normal classification problem.
- 'Local' means labels are only valid inside that window: speaker A in window 1 and speaker A in window 2 may be different people. Fixing that is the job of steps 2 and 3.
- This step also does **voice activity detection** (speech vs non-speech) for free: the {nobody} class.

5. Step 2 - Speaker embeddings (voiceprints)

- For each local speaker in each window, the embedding model (a ResNet34 trained on thousands of speakers, WeSpeaker) turns their speech into a **fixed-length vector** - a voiceprint.
- It is trained so that vectors of the **same person are close** and of different people are far apart (measured by cosine similarity), regardless of what words are said.
- Only frames where that speaker talks **alone** are used, so the voiceprint is not polluted by overlap.
- Note: the 'std(): degrees of freedom is <= 0' warning we saw comes from this step. A local speaker had too few clean frames to compute a standard deviation in the statistics-pooling layer. It is harmless here.

6. Step 3 - Global clustering (VBx + PLDA)

- Now we have many voiceprints across the recording. **Clustering** groups those that belong to the same person; each group becomes a global label (SPEAKER_00, SPEAKER_01 ...).
- **PLDA** (Probabilistic Linear Discriminant Analysis) transforms embeddings so that differences between speakers are emphasised and differences within a speaker (mood, microphone) are reduced.
- **VBx** is a Bayesian clustering method (Variational Bayes over a hidden Markov model) that also estimates how many speakers there are. community-1 uses it instead of the older agglomerative hierarchical clustering of version 3.1, which improved speaker counting.
- If you pass **num_speakers=2** (or min/max_speakers), clustering is constrained to that number - useful for consultations where you know there are exactly two people.

7. Step 4 - Reconstruction and speaker counting

- Each local speaker in each window is mapped to its global cluster.
- Because windows overlap, every frame has several predictions; they are **averaged** (aggregated).
- The segmentation output also gives the **number of active speakers** per frame; the top-N most likely global speakers are kept for that frame.
- Consecutive frames with the same speaker are merged into **turns** (start, end, label). Very short gaps are filled and very short turns may be removed.

8. Regular vs exclusive diarization

```
Regular   :  SPEAKER_00 [=========]
             SPEAKER_01        [=======]        <- overlap: both active 4.0-4.6 s
Exclusive :  SPEAKER_00 [======]
             SPEAKER_01        [=========]      <- only ONE speaker at any moment
```

- **speaker_diarization** (our JSON key 'speakers'): the most faithful output; overlapping turns allowed.
- **exclusive_speaker_diarization** ('exclusive_speakers'): in overlapped regions only the most likely speaker is kept. New in pyannote 4.0, designed for matching with transcripts, because Whisper also produces only one stream of words.
- In our test clip there was no overlap, so both lists are identical.

9. pyannote.audio 4.x vs 3.x

	3.x (speaker-diarization-3.1)	4.x (community-1, ours)
Login argument	use_auth_token='hf_...'	token='hf_...' (or saved login)
Pipeline output	an Annotation object	an object with .speaker_diarization and .exclusive_speaker_diarization
Clustering	agglomerative hierarchical	VBx + PLDA
Audio reading	torchaudio / soundfile	torchcodec (needs ffmpeg) - we bypass it by passing a waveform
Exclusive mode	no	yes
DER on AMI (meetings)	18.8 %	17.0 %

Many online tutorials still use the 3.x API; copying them into 4.x code causes errors. Always check the installed version's documentation.

10. Hugging Face gated models and tokens

- **Gated model**: the files are public but you must log in and accept the authors' conditions (here: share contact info) before downloading. Access is per **account**.
- **Access token**: proves to Hugging Face which account is downloading. A **Read** token can read any gated repo your account has accepted; a **fine-grained** token needs that permission ticked.
- **Our debugging story**: login worked (user wolver789) but downloads returned 403 'not in the authorized list'. The token was a valid Read token, so the cause was that the account had not accepted the conditions. 401 = not authenticated (bad/missing token); 403 = authenticated but not allowed.
- Never hard-code tokens in source code: we use *hf auth login*, which stores it in ~/.cache/huggingface/token (or set the HF_TOKEN environment variable).
- After the first download the model is cached locally and runs **offline**; patient audio is never uploaded.

11. Our code, step by step (diarize.py)

Code	What happens
Pipeline.from_pretrained(PIPELINE_NAME)	Downloads (first time) config + weights, uses the saved token.
pipeline.to(torch.device(device))	Moves models to cpu / mps (Apple GPU) / cuda.
decode_audio(path, sampling_rate=16000)	Decodes with PyAV (same as Whisper) -> float32 array. No ffmpeg needed; both stages see identical samples.
torch.from_numpy(audio).unsqueeze(0)	Shape (1, N): pyannote expects (channels, samples).
pipeline({'waveform': ..., 'sample_rate': 16000}, num_speakers=..., hook=ProgressHook())	Runs segmentation -> embeddings -> clustering, showing progress bars.
annotation.itertracks(yield_label=True)	Iterates (segment, track, label) - converted to JSON dicts.

Files: **diarize.py** (new script) and **requirements-diarization.txt** (pyannote.audio==4.0.7). transcribe.py and requirements.txt were not modified.

12. Reading our output

Diarization (pyannote)	Whisper segment (Stage 1)	Who it really was
SPEAKER_00 0.03 - 2.46	0.00 - 2.08 Good morning, what brings you in today?	Daniel (doctor voice)
SPEAKER_01 2.88 - 6.54	2.84 - 6.42 I have been experiencing fever ...	Samantha (patient voice)
SPEAKER_00 7.07 - 9.35	7.00 - 9.16 Have you taken any medication for it?	Daniel
SPEAKER_01 9.70 - 13.78	9.70 - 13.60 Just paracetamol, 500 milligrams ...	Samantha

- **Correct**: 2 speakers found without being told, and every turn correctly attributed.
- **Boundaries differ by 0.1-0.4 s** between the two models (e.g. 2.08 vs 2.46). They are independent models with different frame resolutions (~17 ms vs 20 ms) and different ideas of where speech fades out. This mismatch is exactly why the next stage, **timestamp alignment**, is needed.
- Speaking time: SPEAKER_00 4.71 s, SPEAKER_01 7.74 s. In real consultations, talk-time ratio is one possible clue for role identification later.
- Speed: 3.27 s of processing for 14 s of audio on CPU; about 26 s total including loading PyTorch and the model.

Caveat: the test clip uses two very different synthetic voices (British male / American female) with silence between turns - an easy case. Real recordings with overlap and similar voices will be harder.

13. Measuring quality: Diarization Error Rate (DER)

DER = (False Alarm + Missed Speech + Speaker Confusion) / Total reference speech time

- **False alarm**: system says someone speaks, but it is silence.
- **Missed speech**: someone speaks, but the system says silence.
- **Confusion**: speech detected, but assigned to the wrong speaker.
- Before scoring, system labels are optimally **mapped** to reference labels (Hungarian algorithm), because label names are arbitrary.
- A **collar** (e.g. 0.25 s) may be ignored around boundaries since exact boundaries are subjective. pyannote reports DER with no collar and overlap included - the strict setting.
- Other metric: **JER** (Jaccard Error Rate) weights each speaker equally, so a quiet speaker's errors count as much as a talkative one's.

```
Example: 100 s of reference speech
  false alarm 3 s + missed 5 s + confusion 4 s = 12 s  ->  DER = 12 %
```

14. Limitations in a clinical setting

- **Overlap and backchannels** ('mm-hm', 'okay') are often missed or assigned to the wrong speaker.
- **Extra people**: a relative, nurse or interpreter adds speakers; forcing num_speakers=2 would then merge them wrongly. Prefer min_speakers/max_speakers when unsure.
- **Single distant microphone** (a phone on the desk) is much harder than headset recordings.
- **Label instability**: labels are per recording and arbitrary; never assume SPEAKER_00 = doctor.
- **Short utterances** carry little voice information, so 'yes'/'no' answers are the most error-prone.
- **Privacy**: voiceprints are biometric data; keep processing local and do not store embeddings unnecessarily.

15. Interview questions and answers

Q1. What is speaker diarization?

Partitioning an audio recording into segments labelled by speaker identity, answering 'who spoke when'. Labels are anonymous (SPEAKER_00 ...), not real identities.

Q2. How is diarization different from speaker identification and from speech recognition?

Identification matches a voice to a known enrolled person; diarization only groups same-voice segments. Speech recognition produces words; diarization produces speaker time intervals without any text.

Q3. Why does a SOAP-note system need diarization?

SOAP notes depend on who said what: symptoms reported by the patient go to Subjective, while the doctor's findings and plan go to Objective/Assessment/Plan. Without speakers, 'I have a fever' could be attributed to the wrong person.

Q4. Which model did you use and why?

pyannote/speaker-diarization-community-1 on pyannote.audio 4.0.7: the current open-source recommendation, lower DER than 3.1 on every benchmark, runs locally (privacy), CC-BY-4.0, and provides exclusive diarization that simplifies alignment with Whisper.

Q5. Explain the pyannote pipeline.

Sliding-window neural segmentation finds local speakers per frame (with overlap) -> an embedding model produces a voiceprint per local speaker -> VBx/PLDA clustering links them into global speakers -> reconstruction aggregates windows and counts active speakers per frame.

Q6. What is a speaker embedding?

A fixed-length vector representing voice characteristics, trained so the same speaker's vectors are close (high cosine similarity) and different speakers' are far apart, independent of the words spoken.

Q7. What is the powerset formulation in segmentation?

Instead of independent per-speaker sigmoid outputs, the model classifies each frame into one of the possible speaker combinations (e.g. 7 classes for up to 3 speakers, max 2 overlapping). It removes the need for a tuned detection threshold and handles overlap naturally.

Q8. Why use sliding windows instead of processing the whole file at once?

Neural segmentation works best on short, fixed-length inputs and memory grows with length. Windows keep it efficient; clustering then links speakers across windows for arbitrary-length audio.

Q9. How does the system know how many speakers there are?

Clustering (VBx) estimates the number of clusters from the embeddings. You can also constrain it with num_speakers, or min_speakers / max_speakers.

Q10. Would you set num_speakers=2 for consultations?

Only if two people are guaranteed. Relatives, nurses or interpreters are common, so min_speakers=2 with a max_speakers bound is safer.

Q11. What is DER? Compute an example.

(false alarm + missed + confusion) / total speech. 2 s FA + 3 s missed + 5 s confusion over 100 s = 10%.

Q12. Why must labels be mapped before computing DER?

Label names are arbitrary: the system's SPEAKER_00 may correspond to the reference's speaker B. An optimal one-to-one mapping (Hungarian algorithm) is found first.

Q13. What is the difference between regular and exclusive diarization?

Regular allows overlapping turns. Exclusive keeps only the most likely speaker at each instant, which matches single-stream transcripts like Whisper's.

Q14. Your diarization and Whisper boundaries differ by up to 0.4 s. Why, and what will you do?

They are independent models with different frame rates and speech-boundary behaviour. The next stage aligns them, e.g. by assigning each word to the speaker whose turn overlaps it most (or contains its midpoint).

Q15. Why might overlapping speech be a problem for the whole pipeline?

Whisper typically transcribes only the dominant speaker, and diarization may miss the second one, so interruptions and backchannels can be lost or misattributed.

Q16. What changed between pyannote 3.x and 4.x?

use_auth_token became token; the pipeline returns an object with speaker_diarization and exclusive_speaker_diarization; community-1 uses VBx clustering; audio is read via torchcodec (ffmpeg).

Q17. How did you avoid installing ffmpeg?

Decoded the audio with PyAV (already used by faster-whisper) and passed pyannote an in-memory waveform dict. Side benefit: both stages process identical samples.

Q18. You got 403 'not in the authorized list' - how did you debug it?

whoami confirmed login, and the token metadata showed a Read token, so authentication was fine (401 would mean bad token). 403 meant authorization: the account had not accepted the gated model's conditions.

Q19. What is PLDA?

Probabilistic Linear Discriminant Analysis: models embeddings as speaker factor + noise, so similarity focuses on between-speaker differences and ignores within-speaker variation.

Q20. How would you speed up diarization?

Run on GPU (cuda, or mps on Apple Silicon), batch embedding extraction, provide num_speakers to simplify clustering, or process long files in parallel chunks.

Q21. What privacy issues does diarization raise?

Voiceprints are biometric personal data. Process locally, avoid persisting embeddings, secure outputs, and follow HIPAA/GDPR for patient recordings.

Q22. How would you evaluate on your own data?

Manually annotate a few consultations in RTTM format, run pyannote.metrics DiarizationErrorRate, and inspect confusion vs missed vs false-alarm to know what to improve.